

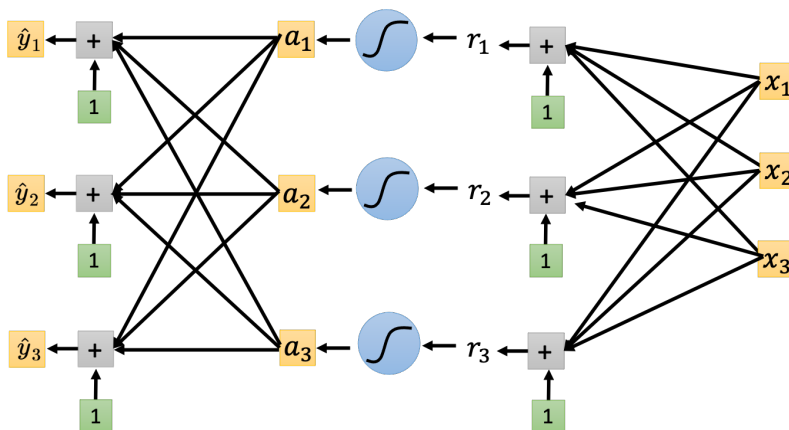


图 3.31 网络多个输出示例

3.6.2 带有 softmax 的分类

按照上述的设定，分类实际过程是：输入 x ，乘上 W ，加上 b ，通过激活函数 σ ，乘上 W' ，再加上 b' 得到向量 $\hat{y}$ 。但实际做分类的时候，往往会把 $\hat{y}$ 通过 softmax 函数得到 y' ，才去计算 y' 跟 $\hat{y}$ 之间的距离。

$$\hat{y} = b' + W' \sigma(b + W x)$$

特征

标签 $y \longleftrightarrow y' = \text{softmax}(\hat{y})$

图 3.32 带有 softmax 的分类

Q: 为什么分类过程中要加上 softmax 函数?

A: 一个比较简单的解释是， y 是独热向量，所以其里面的值只有 0 跟 1，但是 $\hat{y}$ 里面有任何值。既然目标只有 0 跟 1，但 $\hat{y}$ 有任何值，可以先把它归一化到 0 到 1 之间，这样才能跟标签的计算相似度。

softmax 的计算如式 (3.28) 所示，先把所有的 y 取一个指数（负数取指数后也会变成正的），再对其做归一化（除掉所有 y 的指数值的和）得到 y' 。图 3.33 是 softmax 的块 (block)，输入 y_1 、 y_2 和 y_3 ，产生 y'_1 、 y'_2 和 y'_3 。比如 $y_1 = 3$ ， $y_2 = 1$ ， $y_3 = -3$ ，取完指数的时候， $\exp(3) = 20$ 、 $\exp(1) = 2.7$ 和 $\exp(-3) = 0.05$ ，做完归一化后，就变成 0.88、0.12 跟 0。-3 取完指数，再做归一化以后，会变成趋近于 0 的值。所以 softmax 除了归一化，让 y'_1 、 y'_2 和 y'_3 ，变成 0 到 1 之间，和为 1 以外，它还会让大的值跟小的值的差距更大。

$$y'_i = \frac{\exp(y_i)}{\sum_j \exp(y_j)} \quad (3.28)$$

其中， $1 > y'_i > 0$ ， $\sum_i y'_i = 1$ 。

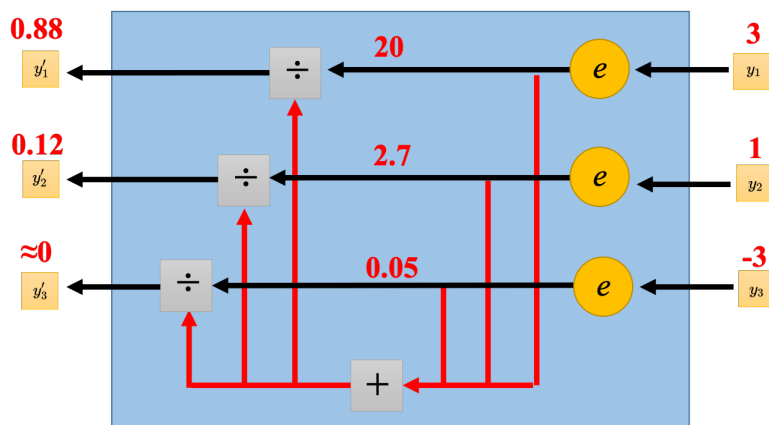


图 3.33 softmax 示例

图 3.33 考虑了三个类的状况，两个类也可以直接套 softmax 函数。但一般有两个类的时候，我们不套 softmax，而是直接取 sigmoid。当只有两个类的时候，sigmoid 和 softmax 是等价的。

3.6.3 分类损失

当我们把 x 输入到一个网络里面产生 $\hat{y}$ 后，通过 softmax 得到 y' ，再去计算 y' 跟 y 之间的距离 e ，如图 3.34 所示。

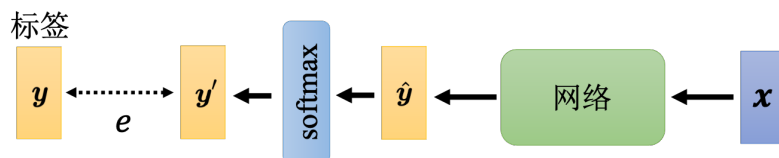


图 3.34 分类损失

计算 y' 跟 y 之间的距离不只一种做法，可以是如式 (3.29) 所示的均方误差，即把 y 里面每一个元素拿出来，计算它们的平方和当作误差。

$$e = \sum_i (y_i - y'_i)^2 \quad (3.29)$$

但如式 (3.30) 所示的交叉熵更常用，当 $\hat{y}$ 跟 y' 相同时，可以最小化交叉熵的值，此时均方误差也是最小的。最小化交叉熵其实就是最大化似然 (maximize likelihood)。

$$e = - \sum_i y_i \ln y'_i \quad (3.30)$$

接下来从优化的角度来说明相较于均方误差，交叉熵是被更常用在分类上。如图 3.35 所示，有一个三类的分类，网络先输出 y_1 、 y_2 和 y_3 ，在通过 softmax 以后，产生 y'_1 、 y'_2 和 y'_3 。假设正确答案是 $[1, 0, 0]^T$ ，要计算 $[1, 0, 0]^T$ 跟 y'_1 、 y'_2 和 y'_3 之间的距离 e ， e 可以是均方误差或交叉熵。假设 y_1 的变化是从 -10 到 10， y_2 的变化也是从 -10 到 10， y_3 就固定设成 -1000。因为 y_3 的值很小，通过 softmax 以后， y'_3 非常趋近于 0，它跟正确答案非常接近，且它对结果影响很少。总之，我们假设 y_3 设一个定值，只看 y_1 跟 y_2 有变化的时候，对损失 e 的影响。

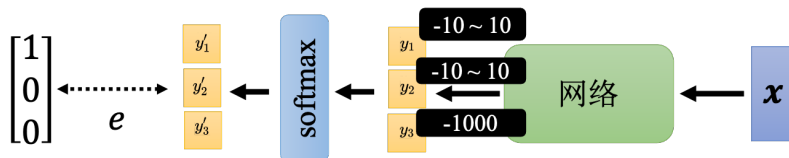


图 3.35 使用 softmax 的好处

图 3.36 是分别在 e 为均方误差和交叉熵时, y_1 、 y_2 的变化对损失的影响, 对误差表面的影响, 红色代表损失大, 蓝色代表损失小。如果 y_1 很大, y_2 很小, 代表 y'_1 会很接近 1, y'_2 会很接近 0。所以不管 e 取均方误差或交叉熵, 如果 y_1 大、 y_2 小, 损失都是小的; 如果 y_1 小, y_2 大, y'_1 是 0, y'_2 是 1, 这个时候损失会比较大。

图 3.36 中左上角损失大, 右下角损失小, 所以期待最后在训练的时候, 参数可以“走”到右下角的地方。假设参数优化开始的时候, 对应的损失都是左上角。如果选择交叉熵, 如图 3.36(a) 所示, 左上角圆圈所在的点有斜率的, 所以可以通过梯度, 一路往右下的地方“走”; 如果选均方误差, 如图 3.36(b) 所示, 左上角圆圈就卡住了, 均方误差在这种损失很大的地方, 它是非常平坦的, 其梯度是非常小趋近于 0 的。如果初始时在圆圈的位置, 离目标非常远, 其梯度又很小, 无法用梯度下降顺利地“走”到右下角。

因此做分类时, 选均方误差的时候, 如果没有好的优化器, 有非常大的可能性会训练不起来。如果用 Adam, 虽然图 3.36(b) 中圆圈的梯度很小, 但 Adam 会自动调大学习率, 还有机会走到右下角, 不过训练的过程比较困难。总之, 改变损失函数可以改变优化的难度。

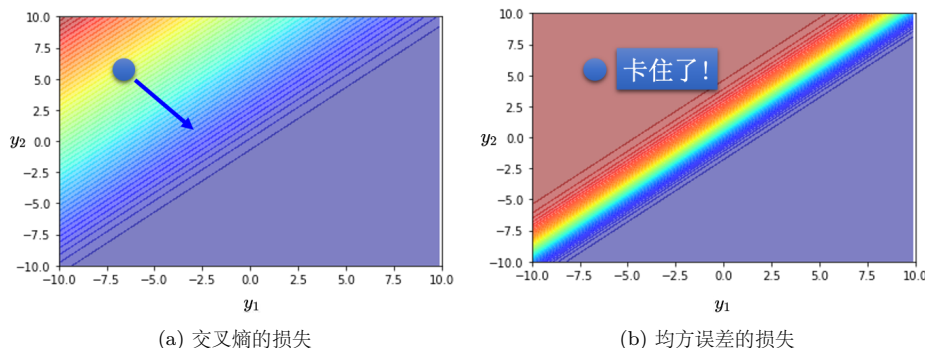


图 3.36 均方误差、交叉熵优化对比

3.7 批量归一化

如果误差表面很崎岖, 它比较难训练。能不能直接改误差表面的地貌, “把山铲平”, 让它变得比较好训练呢? **批量归一化 (Batch Normalization, BN)** 就是其中一个“把山铲平”的想法。不要小看优化这个问题, 有时候就算误差表面是凸 (convex) 的, 它就是一个碗的形状, 都不一定很好训练。如图 3.37 所示, 假设两个参数对损失的斜率差别非常大, 在 w_1 这个方向上面, 斜率变化很小, 在 w_2 这个方向上面斜率变化很大。

如果是固定的学习率, 可能很难得到好的结果, 所以我们才需要自适应的学习率, Adam 等比较进阶的优化的方法, 才能够得到好的结果。从另外一个方向想, 直接把难做的误差表

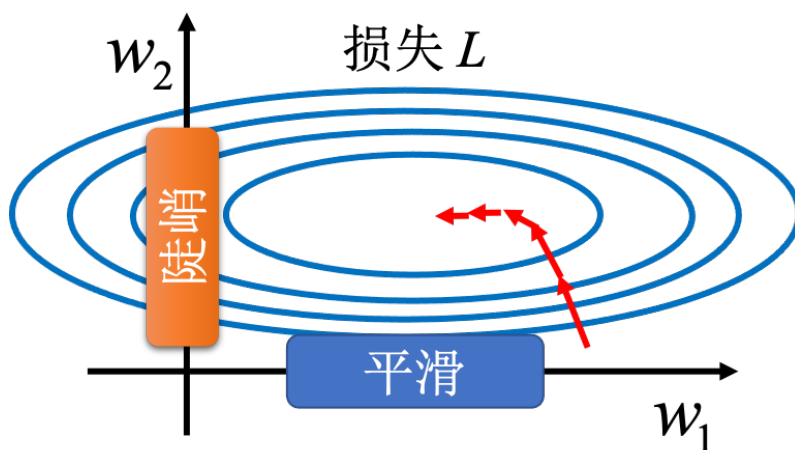


图 3.37 训练的问题

面把它改掉，看能不能够改得好做一点。在做这件事之前，第一个要问的问题就是： w_1 跟 w_2 斜率差很多的这种状况，到底是从什么地方来的。

图 3.38 是一个非常简单的模型，其输入是 x_1 跟 x_2 ，对应的参数为 w_1 跟 w_2 ，它是一个线性的模型，没有激活函数。 w_1 乘 x_1 ， w_2 乘 x_2 加上 b 以后就得到 $\hat{y}$ ，然后会计算 $\hat{y}$ 跟 y 之间的差距当做 e ，把所有训练数据 e 加起来就是损失，然后去最小化损失。

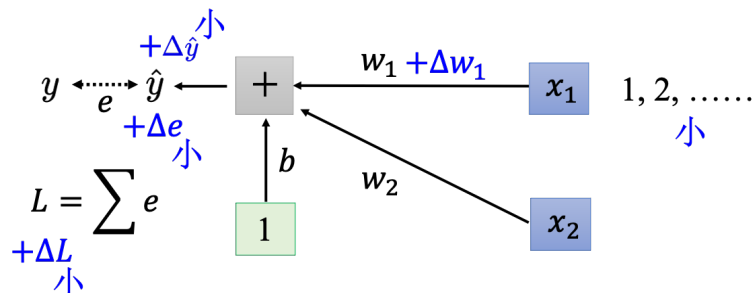


图 3.38 简单的线性模型

什么样的状况会产生像上面这样子，比较不好训练的误差表面呢？对 w_1 有一个小小的改变，比如加上 Δw_1 的时候， L 也会有一个改变，那这个 w_1 呢，是通过 w_1 改变的时候，就改变了 y ， y 改变的时候就改变了 e ，接下来就改变了 L 。

什么时候 w_1 的改变会对 L 的影响很小呢，也就是它在误差表面上的斜率会很小呢？一个可能性是当输入很小的时候，假设 x_1 的值在不同的训练样本里面，它的值都很小，那因为 x_1 是直接乘上 w_1 ，如果 x_1 的值都很小， w_1 有一个变化的时候，它得到的，它对 y 的影响也是小的，对 e 的影响也是小的，它对 L 的影响就会是小的。反之，如图 3.39 所示，如果是 x_2 的话，假设 x_2 的值都很大，当 w_2 有一个小小的变化的时候，虽然 w_2 这个变化可能很小，但是因为它乘上了 x_2 ， x_2 的值很大，那 y 的变化就很大， e 的变化就很大， L 的变化就会很大，就会导致我们在 w 这个方向上，做变化的时候，我们把 w 改变一点点，误差表面就会有很大的变化。所以既然在这个线性的模型里面，当输入的特征，每一个维度的值，它的范围差距很大的时候，我们就可能产生像这样子的误差表面，就可能产生不同方向，斜率非常不同，

坡度非常不同的误差表面所以怎么办呢，有没有可能给特征里面不同的维度，让它有同样的数值的范围。如果我们给不同的维度，同样的数值范围的话，那我们可能就可以制造比较好的误差表面，让训练变得比较容易一点其实有很多不同的方法，这些不同的方法往往就会合起来统称为**特征归一化 (feature normalization)**。

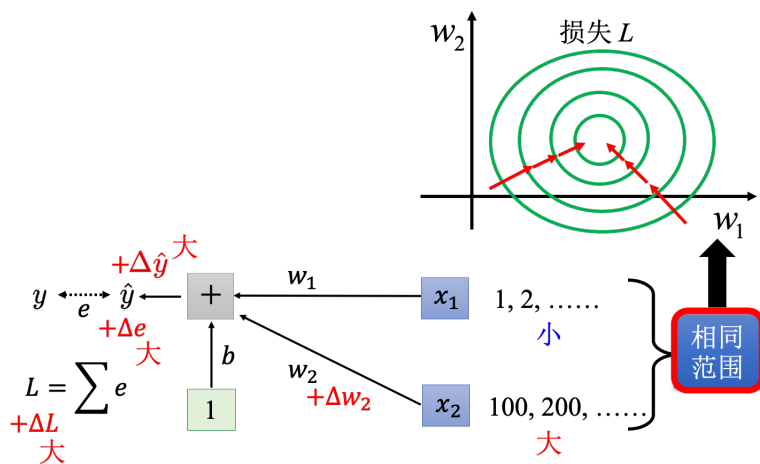


图 3.39 需要特征归一化的原因

以下所讲的方法只是特征归一化的一种可能性，即 Z 值归一化 (Z-score normalization)，也称为标准化 (standardization)。它并不是特征归一化的全部，假设 x^1 到 x^R ，是我们所有的训练数据的特征向量。我们把所有训练数据的特征向量，统统都集合起来。向量 x_1 里面就 x_1^1 代表 x^1 的第一个元素， x_1^2 代表 x^2 的第一个元素，以此类推。我们把不同笔数据即不同特征向量，同一个维度里面的数值，把它取出来，对于每个维度 i ，计算其**平均值 (mean)** m_i 和**标准差 (standard deviation)** σ_i 。接下来我们就可以做一种归一化。

$$\tilde{x}_i^r \leftarrow \frac{x_i^r - m_i}{\sigma_i} \quad (3.31)$$

我们就是把这边的某一个数值 x ，减掉这个维度算出来的平均值，再除掉这个维度，算出来的标准差，得到新的数值 $\tilde{x}$ 。得到新的数值以后，再把新的数值把它塞回去。

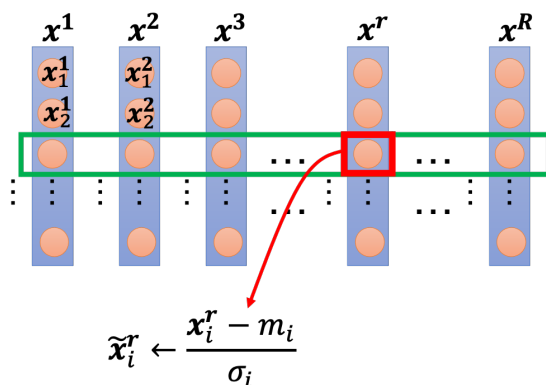


图 3.40 Z 值归一化

归一化有个好处，做完归一化以后，这个维度上面的数值就会平均是 0，其方差是 1，所

以这一排数值的分布就都会在 0 上下；对每一个维度都做一样的归一化，所有特征不同维度的数值都在 0 上下，可能就可以制造一个比较好的误差表面。所以像这样子的特征归一化方式往往对训练有帮助，它可以让在做梯度下降的时候，损失收敛更快一点，训练更顺利一点。

3.7.1 考虑深度学习

$\tilde{x}$ 代表归一化的特征，把它丢到深度网络里面，去做接下来的计算和训练。如图 3.41 所示， $\tilde{x}^1$ 通过第一层得到 z^1 ，有可能通过激活函数，不管是选 sigmoid 或者 ReLU 都可以，再得到 a^1 ，接着再通过下一层等等。对每个 x 都做类似的事情。

虽然 $\tilde{x}$ 已经做归一化了，但是通过 W_1 以后，没有做归一化。如果 $\tilde{x}$ 通过 W_1 得到 z^1 ，而 z^1 不同的维度间，它的数值的分布仍然有很大的差异，训练 W_2 第二层的参数也会有困难。对于 W_2 ， a 或 z 其实也是一种特征，也应该要对这些特征做归一化。如果选择 sigmoid，比较推荐对 z 做特征归一化，因为 sigmoid 是一个 s 的形状，其在 0 附近斜率比较大，如果对 z 做特征归一化，把所有的值都挪到 0 附近，到时候算梯度的时候，算出来的值会比较大。如果使用别的激活函数，可能对 a 归一化也会有好的结果。一般而言，特征归一化，要放在激活函数之前，之后都是可以的，在实现上，没有太大的差别。

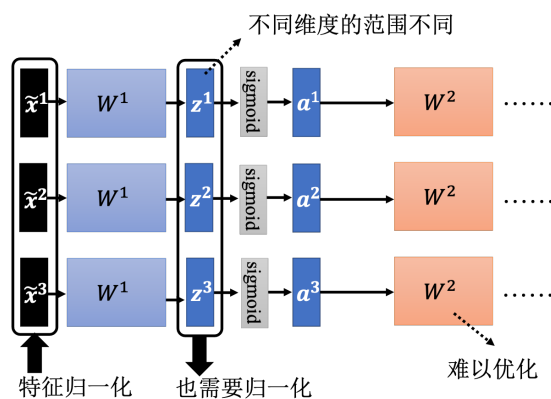


图 3.41 深度学习的归一化

如何对 z 做特征归一化？ z 可以看成另外一种特征。首先计算下 z^1, z^2, z^3 的平均值，即

$$\mu = \frac{1}{3} \sum_{i=1}^3 z^i \quad (3.32)$$

接下来计算标准差

$$\sigma = \sqrt{\frac{1}{3} \sum_{i=1}^3 (z^i - \mu)^2} \quad (3.33)$$

注意，式 (3.33) 中的平方就是指对每一个元素都去做平方，开根号指的是对向量里面的每一个元素开根号。

最后，根据计算出的 μ 和 σ 进行归一化：

$$\tilde{z}^i = \frac{z^i - \mu}{\sigma} \quad (3.34)$$

其中，除号代表逐元素的除，即分子分母两个向量对应元素相除。

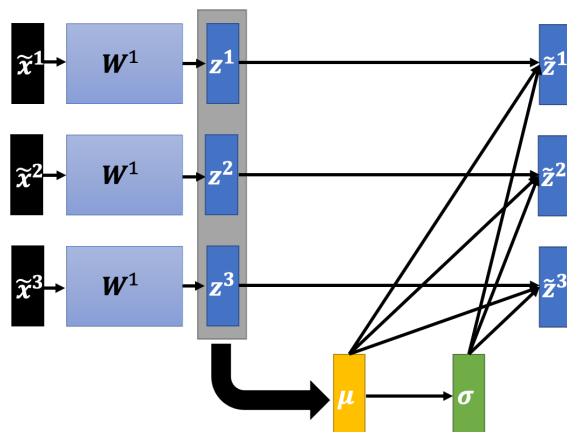


图 3.42 深度学习中间层的特征归一化

归一化的过程如图 3.42 所示。

如图 3.43 所示，接下来可以通过激活函数得到其他向量， μ 跟 σ 都是根据 z^1, z^2, z^3 计算出来的。改变了 z^1 的值， a^1 的值也会改变， μ 和 σ 也会改变。 μ, σ 改后， z^2, a^2, z^3, a^3 的值也会改变。之前的 $\tilde{x}_1, \tilde{x}_2, \tilde{x}_3$ 是独立分开处理的，但是在做特征归一化以后，这三个样本变得彼此关联了。所以有做特征归一化的时候，可以把整个过程当做是网络的一部分。即有一个比较大的网络，该网络吃一堆输入，用这堆输入在这个网络里面计算出 μ, σ ，接下来产生一堆输出。这边就会有一个问题了，因为训练数据非常多，现在一个数据集可能有上百万笔数据，GPU 的显存无法把它整个数据集的数据都加载进去。因此，在实现的时候，我们不会让这个网络考虑整个训练数据里面的所有样本，而是只会考虑一个批量里面的样本。比如批量设 64，这个网络就是把 64 笔数据读进去，计算这 64 笔数据的 μ, σ ，对这 64 笔数据做归一化。因为实际实现的时候，只对一个批量里面的数据做归一化，所以技巧称为批量归一化。一定要有一个够大的批量，才算得出 μ, σ 。所以批量归一化适用于批量大小比较大的时候，批量大小如果比较大，也许这个批量大小里面的数据就足以表示整个数据集的分布。这个时候就不需要对整个数据集做特征归一化，而改成只在一个批量上做特征归一化作为近似。

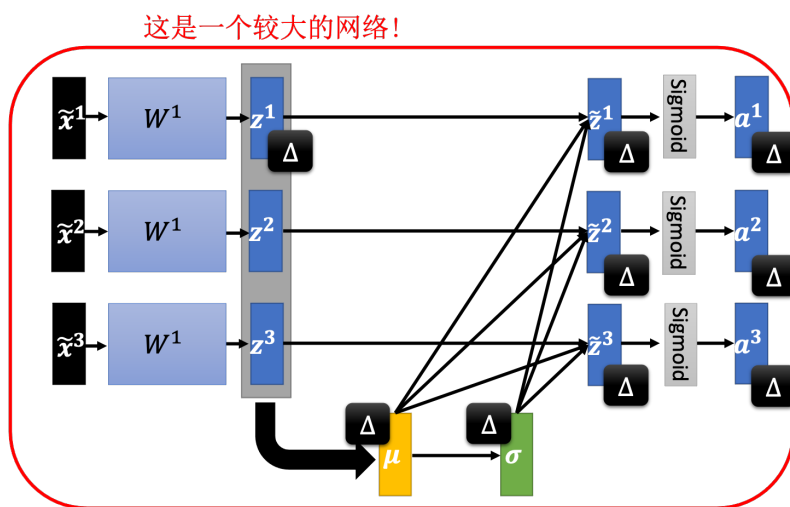


图 3.43 批量归一化可以理解为网络的一部分

在做批量归一化的时候，如图 3.44 所示，往往还会做如下操作：

$$\hat{z}^i = \gamma \odot \tilde{z}^i + \beta \quad (3.35)$$

其中， $\odot$ 代表逐元素的相乘。 β, γ 可以想成是网络的参数，需要另外再被学习出来。

Q：为什么要加上 β 跟 γ 呢？

A：如果做归一化以后， $\tilde{z}$ 的平均值一定是 0，如果平均值是 0 的话，这会给网络一些限制，这个限制可能会带来负面的影响，所以需要把 β, γ 加回去，让网络隐藏层的输出平均值不是 0。让网络学习 β, γ 来调整一下输出的分布，从而来调整 $\hat{z}$ 的分布。

Q：批量归一化是为了要让每一个不同的维度的范围相同，如果把 γ 跟 β 加进去，这样不同维度的分布，其范围不会又都不一样了吗？

A：有可能，但是实际上在训练的时候， γ 的初始值都设为 1，所以 γ 值都为 1 的向量。 β 是值全部都是 0 的向量，即零向量。所以让网络在一开始训练的时候，每一个维度的分布，是比较接近的，也许训练到后来，已经训练够长的一段时间，已经找到一个比较好的误差表面，走到一个比较好的地方以后，再把 γ, β 慢慢地加进去，所以加了 γ, β 的批量归一化，往往对训练是有帮助的。

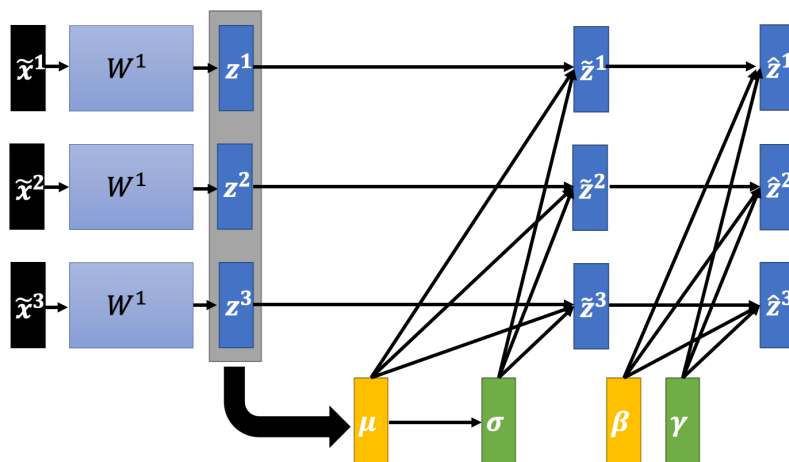


图 3.44 加 γ, β 的批量归一化

3.7.2 测试时的批量归一化

以上说的都是训练的部分，测试有时候又称为**推断 (inference)**。批量归一化在测试的时候，会有什么样的问题呢？在测试的时候，我们一次会得到所有的测试数据，确实也可以在测试的数据上面，制造一个一个批量。但是假设系统上线，做一个真正的线上的应用，比如批量大小设 64，我一定要等 64 笔数据都进来，才做一次做运算，这显然是不行的。

但是在做批量归一化的时候， μ, σ 是用一个批量的数据算出来的。但如果在测试的时候，根本就没有批量，如何算 μ, σ 呢？所以真正的实现上的解法是这个样子的。批量归一化在测试的时候，并不需要做什么特别的处理，PyTorch 已经处理好了。在训练的时候，如果有在做

批量归一化，每一个批量计算出来的 μ, σ ，都会拿出来算移动平均 (moving average)。假设现在有各个批量计算出来的 $\mu^1, \mu^2, \mu^3, \dots, \mu^t$ ，则可以计算移动平均

$$\bar{\mu} \leftarrow p\bar{\mu} + (1-p)\mu^t \quad (3.36)$$

其中， $\bar{\mu}$ 是 μ 的个平均值， p 是因子，这也是一个常数，这也是一个超参数，也是需要调的那种。在 PyTorch 里面， p 设 0.1。计算滑动平均来更新 μ 的平均值。最后在测试的时候，就不用算批量里面的 μ 跟 σ 了。因为测试的时候，在真正应用上也没有批量，就可以就直接拿 $\bar{\mu}$ 跟 $\bar{\sigma}$ 来取代原来的 μ 跟 σ ，如图 3.45 所示，这就是批量归一化在测试的时候的运作方式。

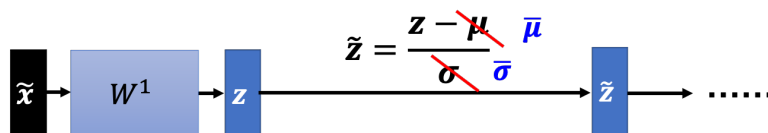


图 3.45 测试时的批量归一化

图 3.46 是从批量归一化原始文献的实验结果，横轴代表的是训练的过程，纵轴代表的是验证集上的准确率。黑色的虚线是没有做批量归一化的结果，它用的是 inception 的网络（一种网络以 CNN 为基础的网络架构）。如果有做批量归一化，则是红色的这一条虚线。红色虚线的训练速度显然比黑色的虚线还要快很多。虽然只要给模型足够的训练的时间，最后会收敛都差不多的准确率。但是红色虚线可以在比较短的时间内跑到一样的准确率。蓝色的菱形代表说几个点的准确率是一样的。粉色的线是 sigmoid 函数，一般的认知，但一般都会选择 ReLU，而不是用 sigmoid 函数，因为 sigmoid 函数的训练是比较困难的。但是这边想要强调的点是，就算是 sigmoid 比较难搞的加批量归一化，还是可以训练的，这边没有 sigmoid，没有做批量归一化的结果。因为在这个实验上，sigmoid 不加批量归一化，根本连训练都训练不起来。蓝色的实线跟这个蓝色的虚线呢是把学习率设比较大一点， $\times 5$ 就是学习率变原来的 5 倍； $\times 30$ 就是学习率变原来的 30 倍。因为如果做批量归一化，误差表面会比较平滑，比较容易训练，所以就可以把学习率设大一点。这边有个不好解释的地方，学习率设 30 倍的时候比 5 倍差，作者也没有解释。

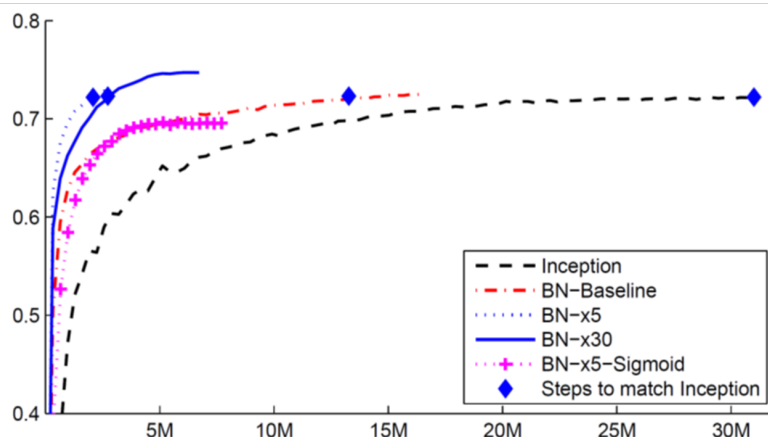


图 3.46 批量归一化实验结果^[10]

3.7.3 内部协变量偏移

接下来的问题就是批量归一化为什么会有帮助呢？原始的批量归一化论文里面提出**内部协变量偏移 (internal covariate shift)** 概念。如图 3.47 所示，假设网络有很多层， x 通过第一层后得到 a ， a 通过第二层以后得到 b ；计算出梯度以后，把 A 更新成 A' ，把 B 这一层的参数更新成 B' 。但是作者认为说，我们在计算 B 更新到 B' 的梯度的时候，这个时候前一层的参数是 A ，或者是前一层的输出是 a 。那当前一层从 A 变成 A' 的时候，其输出就从 a 变成 a' 。但是我们计算这个梯度的时候，是根据 a 算出来，所以这个更新的方向也许它适合用在 a 上，但不适合用在 a' 上面。因为我们每次都有做批量归一化，就会让 a 和 a' 的分布比较接近，也许这样就会对训练有帮助。但是论文“How Does Batch Normalization Help Optimization?”^[11] 认为内部协变量偏移有问题。这篇论文从不同的角度来说明内部协变量偏移不一定是训练网络的时候的一个问题。批量归一化会比较好，可能不一定是因为它解决了内部协变量偏移。这篇论文里面做了很多实验，比如其比较了训练的时候 a 的分布的变化，发现不管有没有做批量归一化，其变化都不大。就算是变化很大，对训练也没有太大的伤害。不管是根据 a 算出来的梯度，还是根据 a' 算出来的梯度，方向居然都差不多。内部协变量偏移可能不是训练网络的时候，最主要的问题，它可能也不是批量归一化会好的一个的关键。

协变量偏移 (covariate shift)，训练集和预测集样本分布不一致的问题就叫做协变量偏移现象，这个词汇是原来就有的，内部协变量偏移是批量归一化的作者自己发明的。

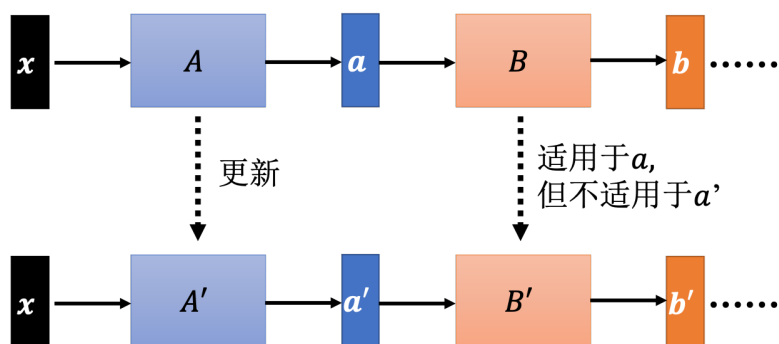


图 3.47 内部协变量偏移示例

为什么批量归一化会比较好呢，那在这篇“How Does Batch Normalization Help Optimization?”这篇论文从实验和理论上，至少支持批量归一化可以改变误差表面，让误差表面比较不崎岖这个观点。所以这个观点是有理论的支持，也有实验的佐证的。如果要想让网络误差表面变得比较不崎岖，其实不一定要做批量归一化，还有很多其他的方法都可以让误差表面变得不崎岖，这篇论文就试了一些其他的方法，发现跟批量归一化表现也差不多，甚至还稍微好一点，这篇论文的作者也觉得批量归一化是一种偶然的发现，但无论如何，其是一个有用的方法。其实批量归一化不是唯一的归一化，还有很多归一化方法，比如批量重归一化 (batch renormalization)^[12]、层归一化 (layer normalization)^[13]、实例归一化 (instance normalization)^[14]、组归一化 (group normalization)^[15]、权重归一化 (weight normalization)^[16] 和谱归一化 (spectrum normalization)^[17]。

参考文献

- [1] KESKAR N S, MUDIGERE D, NOCEDAL J, et al. On large-batch training for deep learning: Generalization gap and sharp minima[J]. arXiv preprint arXiv:1609.04836, 2016.
- [2] GUPTA V, SERRANO S A, DECOSTE D. Stochastic weight averaging in parallel: Large-batch training that generalizes well[J]. arXiv preprint arXiv:2001.02312, 2020.
- [3] YOU Y, GITMAN I, GINSBURG B. Large batch training of convolutional networks[J]. arXiv preprint arXiv:1708.03888, 2017.
- [4] YOU Y, LI J, REDDI S, et al. Large batch optimization for deep learning: Training bert in 76 minutes[J]. arXiv preprint arXiv:1904.00962, 2019.
- [5] AKIBA T, SUZUKI S, FUKUDA K. Extremely large minibatch sgd: Training resnet-50 on imagenet in 15 minutes[J]. arXiv preprint arXiv:1711.04325, 2017.
- [6] GOYAL P, DOLLÁR P, GIRSHICK R, et al. Accurate, large minibatch sgd: Training imagenet in 1 hour[J]. arXiv preprint arXiv:1706.02677, 2017.
- [7] KINGMA D P, BA J. Adam: A method for stochastic optimization[J]. arXiv preprint arXiv:1412.6980, 2014.
- [8] HE K, ZHANG X, REN S, et al. Deep residual learning for image recognition[C]//Proceedings of the IEEE conference on computer vision and pattern recognition. 2016: 770-778.
- [9] LIU L, JIANG H, HE P, et al. On the variance of the adaptive learning rate and beyond [J]. arXiv preprint arXiv:1908.03265, 2019.
- [10] IOFFE S, SZEGEDY C. Batch normalization: Accelerating deep network training by reducing internal covariate shift[C]//International conference on machine learning. pmlr, 2015: 448-456.
- [11] SANTURKAR S, TSIPRAS D, ILYAS A, et al. How does batch normalization help optimization?[J]. Advances in neural information processing systems, 2018, 31.
- [12] IOFFE S. Batch renormalization: Towards reducing minibatch dependence in batch-normalized models[J]. Advances in neural information processing systems, 2017, 30.
- [13] BA J L, KIRO S J R, HINTON G E. Layer normalization[J]. arXiv preprint arXiv:1607.06450, 2016.
- [14] ULYANOV D, VEDALDI A, LEMPITSKY V. Instance normalization: The missing ingredient for fast stylization[J]. arXiv preprint arXiv:1607.08022, 2016.
- [15] WU Y, HE K. Group normalization[C]//Proceedings of the European conference on computer vision (ECCV). 2018: 3-19.

- [16] SALIMANS T, KINGMA D P. Weight normalization: A simple reparameterization to accelerate training of deep neural networks[J]. Advances in neural information processing systems, 2016, 29.
- [17] YOSHIDA Y, MIYATO T. Spectral norm regularization for improving the generalizability of deep learning[J]. arXiv preprint arXiv:1705.10941, 2017.

第 4 章 卷积神经网络

我们从卷积神经网络开始,探讨网络的架构设计。卷积神经网络是一种非常典型的网络架构,常用于图像分类等任务。通过卷积神经网络,我们可以知道网络架构如何设计,以及为什么合理的网络架构可以优化网络的表现。

所谓图像分类,就是给机器一张图像,由机器去判断这张图像里面有什么样的东西——是猫还是狗、是飞机还是汽车。怎么把图像当做模型的输入呢?对于机器,图像可以描述为三维张量(张量可以想成维度大于 2 的矩阵)。一张图像是一个三维的张量,其中一维代表图像的宽,另外一维代表图像的高,还有一维代表图像的**通道(channel)**的数目。

Q: 什么是通道?

A: 彩色图像的每个像素都可以描述为红色(red)、绿色(green)、蓝色(blue)的组合,这 3 种颜色就称为图像的 3 个色彩通道。这种颜色描述方式称为 RGB 色彩模型,常用于在屏幕上显示颜色。

网络的输入往往是向量,因此,将代表图像的三维张量“丢”到网络里之前,需要先将它“拉直”,如图 4.1 所示。在这个例子里面,张量有 $100 \times 100 \times 3$ 个数字,所以一张图像是由 $100 \times 100 \times 3$ 个数字所组成的,把这些数字排成一排就是一个巨大的向量。这个向量可以作为网络的输入,而这个向量里面每一维里面存的数值是某一个像素在某一个通道下的颜色强度。

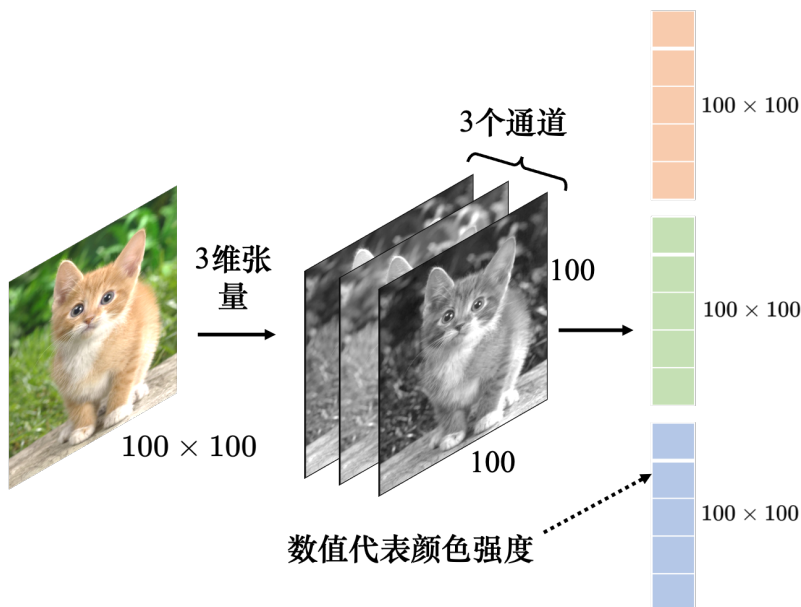


图 4.1 把图像作为模型输入

图像有大有小,而且不是所有图像尺寸都是一样的。常见的处理方式是把所有图像先调整成相同尺寸,再“丢”到图像的识别系统里面。以下的讨论中,默认模型输入的图像尺寸固定为 100 像素 × 100 像素。

如图 4.2 所示,如果把向量当做全连接网络的输入,输入的特征向量(feature vector)的

长度就是 $100 \times 100 \times 3$ 。这是一个非常长的向量。由于每个神经元跟输入的向量中的每个数值都需要一个权重，所以当输入的向量长度是 $100 \times 100 \times 3$ ，且第 1 层有 1000 个神经元时，第 1 层的权重就需要 $1000 \times 100 \times 100 \times 3 = 3 \times 10^7$ 个权重，这是一个非常巨大的数目。更多的参数为模型带来了更好的弹性和更强的能力，但也增加了过拟合的风险。模型的弹性越大，就越容易过拟合。为了避免过拟合，在做图像识别的时候，考虑到图像本身的特性，并不一定需要全连接，即不需要每个神经元跟输入的每个维度都有一个权重。接下来就是针对图像识别这个任务，对图像本身特性进行一些观察。

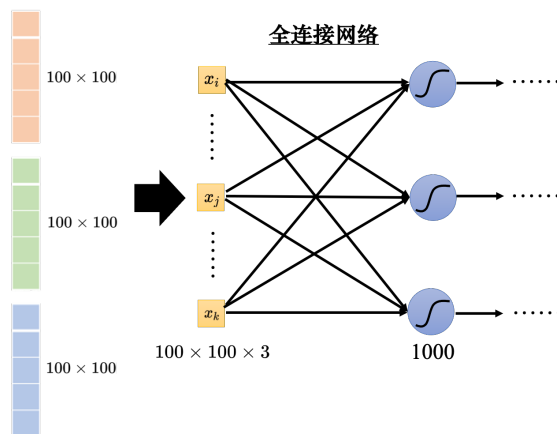


图 4.2 全连接网络

模型的输出应该是什么呢？模型的目标是分类，因此可将不同的分类结果表示成不同的独热向量 y' 。在这个独热向量里面，类别对应的值为 1，其余类别对应的值为 0。例如，我们规定向量中的某些维度代表狗、猫、树等分类结果，那么若分类结果为猫，则猫所对应的维度的数值就是 1，其他东西所对应的维度的数值就是 0，如图 4.3 所示。独热向量 y' 的长度决定了模型可以识别出多少不同种类的东西。如果向量的长度是 5，代表模型可以识别出 5 种不同的东西。现在比较强的图像识别系统往往可以识别出 1000 种以上，甚至上万种不同的东西。如果希望图像识别系统可以识别上万种目标，标签就会是维度上万的独热向量。模型的输出通过 softmax 以后，输出是 $\hat{y}$ 。我们希望 y' 和 $\hat{y}$ 的交叉熵越小越好。

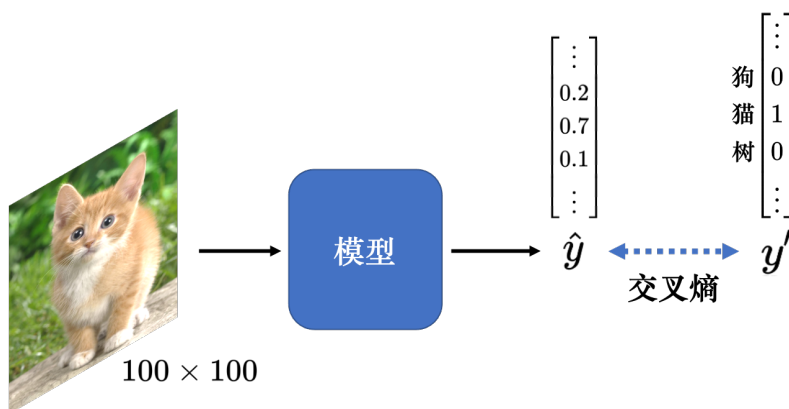


图 4.3 图像分类

4.1 观察 1: 检测模式不需要整张图像

假设我们的任务是让网络识别出图像的动物。对一个图像识别的类神经网络里面的神经元而言，它要做的就是检测图像里面有没有出现一些特别重要的模式 (pattern)，这些模式是代表了某种物体的。比如有三个神经元分别看到鸟嘴、眼睛、鸟爪 3 个模式，这就代表类神经网络看到了一只鸟，如图 4.4 所示。

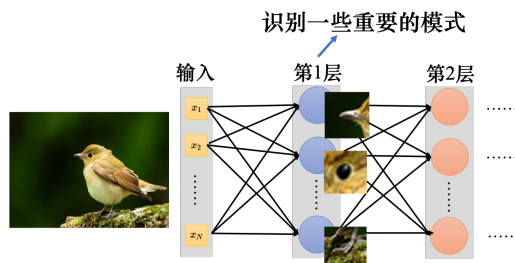


图 4.4 使用神经网络来检测模式

人在判断一个物体的时候，往往也是抓最重要的特征。看到这些特征以后，就会直觉地看到了某种物体。对于机器，也许这是一个有效的判断图像中物体的方法。但假设用神经元来判断某种模式是否出现，也许并不需要每个神经元都去看一张完整的图像。因为并不需要看整张完整的图像才能判断重要的模式（比如鸟嘴、眼睛、鸟爪）是否出现，如图 4.5 所示，要知道图像有没有一个鸟嘴，只要看非常小的范围。这些神经元不需要把整张图像当作输入，只需要把图像的一小部分当作输入，就足以让它们检测某些特别关键的模式是否出现，这是第 1 个观察。

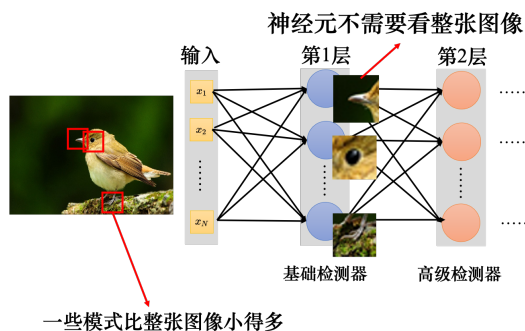


图 4.5 检测模式不需要整张图像

4.2 简化 1: 感受野

根据观察 1 可以做第 1 个简化，卷积神经网络会设定一个区域，即**感受野 (receptive field)**，每个神经元都只关心自己的感受野里面发生的事情，感受野是由我们自己决定的。比如在图 4.6 中，蓝色的神经元的守备范围就是红色正方体框的感受野。这个感受野里面有 $3 \times 3 \times 3$ 个数值。对蓝色的神经元，它只需要关心这个小范围，不需要在意整张图像里面有什么东西，只在意它自己的感受野里面发生的事情就好。这个神经元会把 $3 \times 3 \times 3$ 的数值“拉直”变成一个长度是 $3 \times 3 \times 3 = 27$ 维的向量，再把这 27 维的向量作为神经元的输入，这个神

神经元会给 27 维的向量的每个维度一个权重，所以这个神经元有 $3 \times 3 \times 3 = 27$ 个权重，再加上偏置 (bias) 得到输出。这个输出再送给下一层的神经元当作输入。

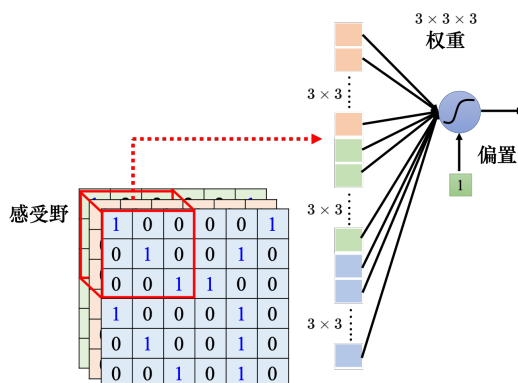


图 4.6 感受野

如图 4.7 所示，蓝色的神经元看左上角这个范围，这是它的感受野。黄色的神经元看右下角 $3 \times 3 \times 3$ 的范围。图 4.7 中的一个正方形代表 $3 \times 3 \times 3$ 的范围，右下角的正方形是黄色神经元的感受野。感受野彼此之间也可以是重叠的，比如绿色的神经元的感受野跟蓝色的、黄色的神经元都有一些重叠的空间。我们没有办法检测所有的模式，所以同个范围可以有多个不同的神经元，即多个神经元可以去守备同一个感受野。接下来我们讨论下如何设计感受野。

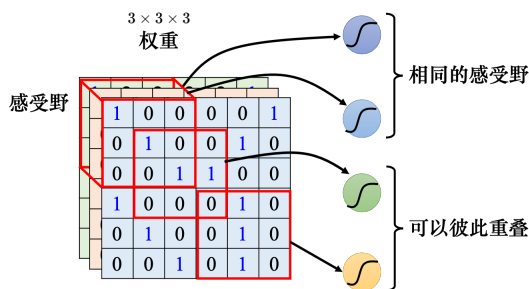


图 4.7 感受野彼此重叠

感受野可以有大有小，因为模式有的比较小，有的比较大。有的模式也许在 3×3 的范围内就可以被检测出来，有的模式也许要 11×11 的范围才能被检测出来。此外，感受野可以只考虑某些通道。目前感受野是 RGB 三个通道都考虑，但也许有些模式只在红色或蓝色的通道会出现，即有的神经元可以只考虑一个通道。之后在讲到网络压缩的时候，会讲到这种网络的架构。感受野不仅可以是正方形的，例如刚才举的例子里面 3×3 、 11×11 ，也可以是长方形的，完全可以根据对问题的理解来设计感受野。虽然感受野可以任意设计，但下面要跟大家讲一下最经典的感受野安排方式。

Q: 感受野一定要相连吗?

A: 感受野的范围不一定要相连, 理论上可以有一个神经元的感受野就是图像的左上角跟右上角。但是就要想想为什么要这么做, 会不会有什么模式也要看一个图像的左上角跟右下角才能够找到。如果没有, 这种感受野就没什么用。要检测一个模式, 这个模式就出现在整个图像里面的某一个位置, 而不是分成好几部分, 出现在图像里面的不同的位置。所以通常的感受野都是相连的领地, 但如果要设计很奇怪的感受野去解决很特别的问题, 完全是可以的, 这都是自己决定的。

一般在做图像识别的时候, 可能不会觉得有些模式只出现在某一个通道里面, 所以会看全部的通道。既然会看全部的通道, 那么在描述一个感受野的时候, 只要讲它的高跟宽, 不用讲它的深度, 因为它的深度就等于通道数, 而高跟宽合起来叫做核大小。图 4.8 中的核大小就是 3×3 。在图像识别里面, 一般核大小不会设太大, 3×3 的核大小就足够了, 7×7 、 9×9 算是蛮大的核大小。如果核大小都是 3×3 , 意味着我们觉得在做图像识别的时候, 重要的模式都只在 3×3 这么小的范围内就可以被检测出来了。但有些模式也许很大, 也许 3×3 的范围没办法检测出来, 后面我们会再回答这个问题。常见的感受野设定方式就是核大小为 3×3 。

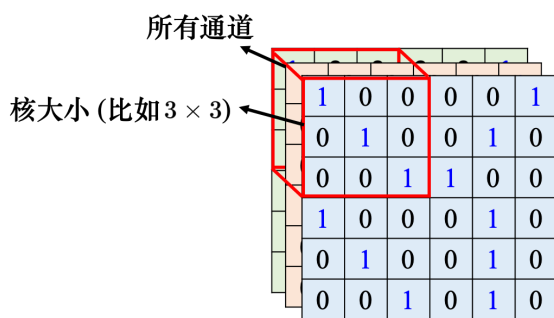


图 4.8 卷积核

一般同一个感受野会有一组神经元去守备这个范围, 比如 64 个或者是 128 个神经元去守备一个感受野的范围。目前为止, 讲的都是一个感受野, 接下来介绍下各个不同感受野之间的关系。我们把左上角的感受野往右移一个步幅, 就制造出一个新的守备范围, 即新的感受野。移动的量称为步幅 (stride), 图 4.9 中的这个例子里面, 步幅就等于 2。步幅是一个超参数, 需要人为调整。因为希望感受野跟感受野之间是有重叠的, 所以步幅往往不会设太大, 一般设为 1 或 2。

Q: 为什么希望感受野之间是有重叠的呢?

A: 因为假设感受野完全没有重叠, 如果有一个模式正好出现在两个感受野的交界上面, 就没有任何神经元去检测它, 这个模式可能会丢失, 所以希望感受野彼此之间有高度的重叠。如令步幅 = 2, 感受野就会重叠。

接下来需要考虑一个问题: 感受野超出了图像的范围, 怎么办呢? 如果不在超过图像的范围“摆”感受野, 就没有神经元去检测出现在边界的模式, 这样就会漏掉图像边界的地方, 所以一般边界的地方也会考虑的。如图 4.10 所示, 超出范围就做填充 (padding), 填充就是补

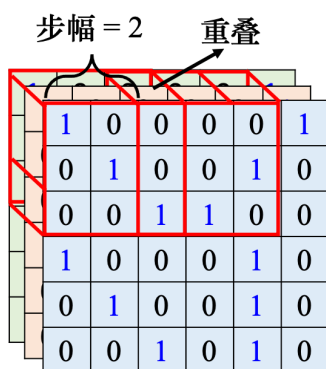


图 4.9 步幅

值，一般使用零填充 (zero padding)，超出范围就补 0，如果感受野有一部分超出图像的范围之外，就当做那个里面的值都是 0。其实也有别的补值的方法，比如补整张图像里面所有值的平均值或者把边界的这些数字拿出来补没有值的地方。

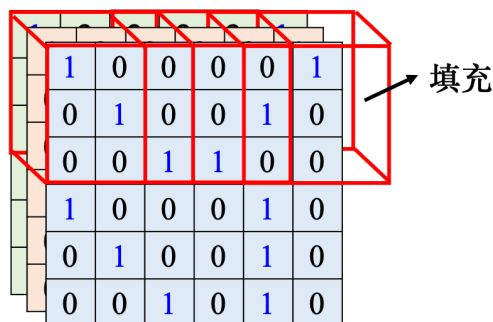


图 4.10 填充

除了水平方向的移动，也会有垂直方向上的移动，垂直方向步幅也是设 2，如图 4.11 所示。我们就按照这个方式扫过整张图像，所以整张图像里面每一寸土地都是有被某一个感受野覆盖的。也就是图像里面每个位置都有一群神经元在检测那个地方，有没有出现某些模式。这个是第 1 个简化。

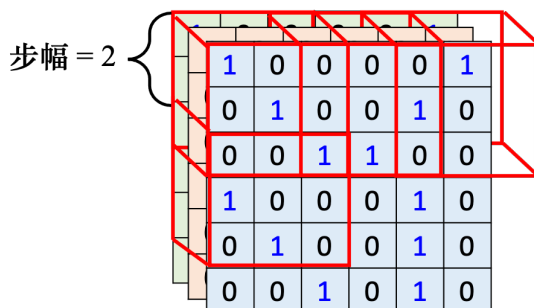


图 4.11 垂直移动

4.3 观察 2: 同样的模式可能会出现在图像的不同区域

第 2 个观察是同样的模式, 可能会出现在图像的不同区域。比如说模式鸟嘴, 它可能出现在图像的左上角, 也可能出现在图像的中间, 同样的模式出现在图像的不同位置也不是太大的问题。如图 4.12 所示, 因为出现在左上角的鸟嘴, 它一定落在某一个感受野里面。因为感受野是盖满整个图像的, 所以图像里面所有地方都在某个神经元的守备范围内。假设在某个感受野里面, 有一个神经元的工作就是检测鸟嘴, 鸟嘴就会被检测出来。所以就算鸟嘴出现在中间也没有关系。假设其中有一个神经元可以检测鸟嘴, 鸟嘴出现在图像的中间也会被检测出来。但这些检测鸟嘴的神经元做的事情是一样的, 只是它们守备的范围不一样。既然如此, 其实没必要每个守备范围都去放一个检测鸟嘴的神经元。如果不同的守备范围都要有一个检测鸟嘴的神经元, 参数量会太多了, 因此需要做出相应的简化。

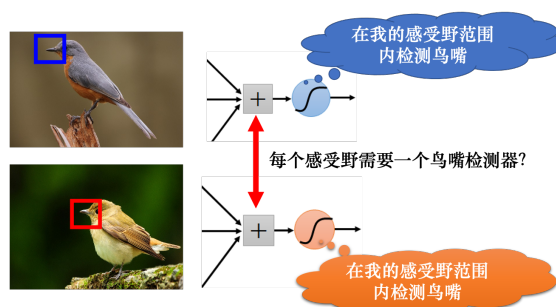


图 4.12 每个感受野都放一个鸟嘴检测器

4.4 简化 2: 共享参数

在提出简化技巧前, 我们先举个类似的例子。这个概念就类似于教务处希望可以推大型的课程一样, 假设每个院系都需要深度学习相关的课程, 没必要在每个院系都开机器学习的课程, 可以开一个比较大型的课程, 让所有院系的人都可以修课。如果放在图像处理上, 则可以让不同感受野的神经元共享参数, 也就是做**参数共享 (parameter sharing)**, 如图 4.13 所示。所谓参数共享就是两个神经元的权重完全是一样的。

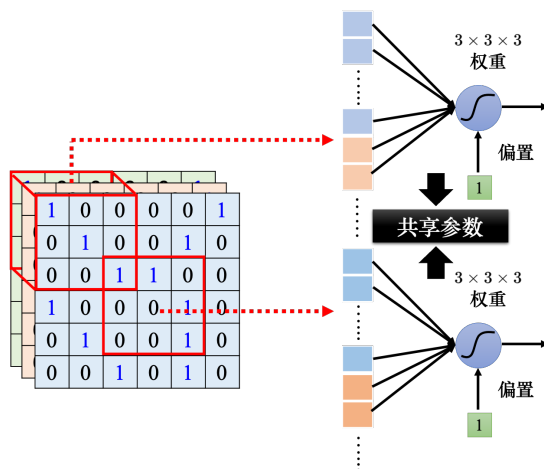


图 4.13 共享参数

如图 4.14 所示, 颜色相同, 权重完全是一样的, 比如上面神经元的第 1 个权重是 w_1 , 下面神经元的第 1 个权重也是 w_1 , 它们是同一个权重, 用同一种颜色黄色来表示。上面神经元跟下面神经元守备的感受野是不一样的, 但是它们的参数是相同的。虽然两个神经元的参数是一模一样, 但它们的输出不会永远都是一样的, 因为它们的输入是不一样的, 它们照顾的范围是不一样的。上面神经元的输入是 $x_1, x_2, \dots$, 下面神经元的输入是 $x'_1, x'_2, \dots$ 。上面神经元的输出为

$$\sigma(w_1x_1 + w_2x_2 + \dots + 1) \quad (4.1)$$

下面神经元的输出为

$$\sigma(w_1x'_1 + w_2x'_2 + \dots + 1) \quad (4.2)$$

因为输入不一样的关系, 所以就算是两个神经元共用参数, 它们的输出也不会是一样的。所以这是第 2 个简化, 让一些神经元可以共享参数, 共享的方式完全可以自己决定。接下来将介绍图像识别方面, 常见的共享方法是如何设定的。

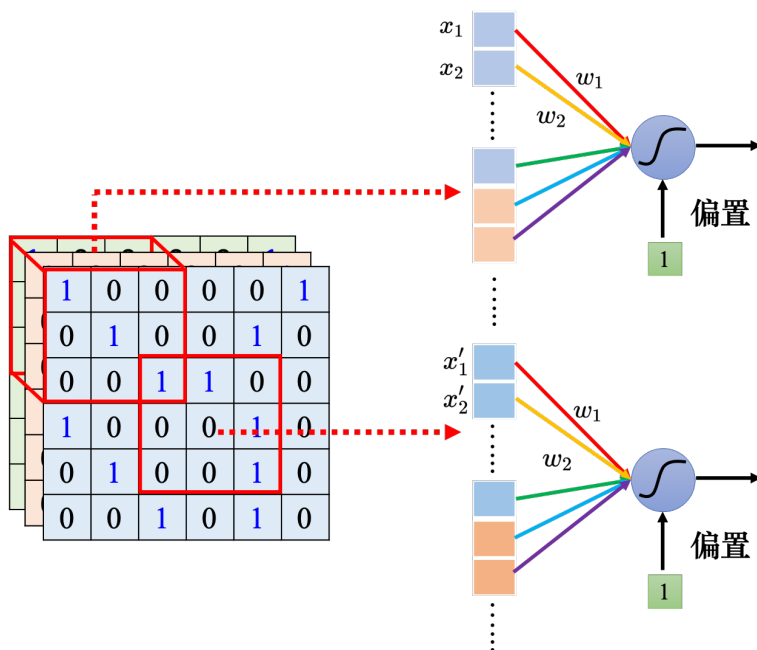


图 4.14 两个神经元共享参数

如图 4.15 所示, 每个感受野都有一组神经元在负责守备, 比如 64 个神经元, 它们彼此之间可以共享参数。图 4.16 中使用一样的颜色代表这两个神经元共享一样的参数, 所以每个感受野都只有一组参数, 就是上面感受野的第 1 个神经元会跟下面感受野的第 1 个神经元共用参数, 上面感受野的第 2 个神经元跟下面感受野的第 2 个神经元共用参数 $\dots$ 所以每个感受野都只有一组参数而已, 这些参数称为**滤波器 (filter)**。这是第 2 个简化的方法。

4.5 简化 1 和 2 的总结

目前已经讲了两个简化的方法, 我们来总结下。如图 4.17 所示, 全连接网络是弹性最大的。全连接网络可以决定它看整张图像还是只看一个范围, 如果它只想看一个范围, 可以把很多权重设成 0。全连接层 (fully-connected layer,) 可以自己决定看整张图像还是一